Predicting Floods in Sub-Saharan Africa Using Satellite Imagery and Computer Vision

Project Objective

Developing a machine learning system that can predict floods 3 days in advance in Sub-Saharan Africa using Satellite imagery and related environmental data. The focus will be on demonstrating the application of computer vision techniques to extract spatial patterns predictive of flooding

Motivation

- Floods are a major hazard in Sub-Saharan Africa, causing property damage, displacement, and loss of life.
- Early prediction enables disaster preparedness and humanitarian response.
- Satellite imagery is increasingly available, offering a rich visual dataset for flood prediction.
- This project demonstrates the use of computer vision pipelines in a real-world social-impact context.

```
# Load necessary libraries

# Core
import os
import glob
from pathlib import Path

import numpy as np
import pandas as pd

# Visualization
import matplotlib.pyplot as plt
import seaborn as sns
import geemap

# Geospatial + raster
import rasterio
from rasterio.plot import show
```

```
import geopandas as gpd
import ee
# ml4floods not installed in this env
# from ml4floods import ...
# from ml4floods.data import worldfloods
# georeader not installed in this env
# from georeader import window_utils
from shapely.geometry import box


# Notebook display
from IPython.display import display

sns.set_theme(style="whitegrid")
plt.rcParams["figure.figsize"] = (10, 6)

# Local fallback for band ordering (used by Dataset)
BANDS_S2 = ['B1','B2','B3','B4','B5','B6','B7','B8','B8A','B9','B10','B1

print("Imports loaded.")
```

```
Imports loaded.
```

```
import torch

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Using device: {device}")
```

```
Using device: cpu
```

```
try:
    from google.colab import drive
    drive.mount('/content/drive')
    public_folder = '/content/drive/My Drive/Public WorldFloods Dataset'
    assert os.path.exists(public_folder), "Add a shortcut to the publice
    google_colab = True
except ImportError as e:
    print(e)
    print("Setting google colab to false, it will need to install the gd
    public_folder = '.'
    google_colab = False
```

```
Mounted at /content/drive
```

```
# Delete the hf dataset that had been downloaded
```

```
import shutil
import os

hf_dataset_cache_dir = os.path.expanduser('~/.cache/huggingface/datasets

print("Disk usage before deletion:")
total, used, free = shutil.disk_usage('/')
print(f"Total: {total / (1024**3):.2f} GB")
print(f"Used: {used / (1024**3):.2f} GB")
print(f"Free: {free / (1024**3):.2f} GB")

if os.path.exists(hf_dataset_cache_dir):
    print(f"\nDeleting Hugging Face datasets cache directory: {hf_datase
    shutil.rmtree(hf_dataset_cache_dir)
    print("Hugging Face datasets cache deleted successfully.")
else:
    print(f"\nHugging Face datasets cache directory not found at {hf_dat

print("\nDisk usage after deletion:")
total, used, free = shutil.disk_usage('/')
print(f"Total: {total / (1024**3):.2f} GB")
print(f"Used: {used / (1024**3):.2f} GB")
print(f"Free: {free / (1024**3):.2f} GB")
```

```
Disk usage before deletion:
Total: 107.72 GB
Used: 20.89 GB
Free: 86.81 GB

Hugging Face datasets cache directory not found at /root/.cache/huggingfa

Disk usage after deletion:
Total: 107.72 GB
Used: 20.89 GB
Free: 86.81 GB
```

```
import shutil
import os

hf_dataset_cache_dir = os.path.expanduser('~/.cache/huggingface/datasets
hf_hub_cache_dir = os.path.expanduser('~/.cache/huggingface/hub')

print("Disk usage before deleting all Hugging Face caches:")
total, used, free = shutil.disk_usage('/')
print(f"Total: {total / (1024**3):.2f} GB")
print(f"Used: {used / (1024**3):.2f} GB")
print(f"Free: {free / (1024**3):.2f} GB")

# Delete datasets cache
if os.path.exists(hf_dataset_cache_dir):
    print(f"\nDeleting Hugging Face datasets cache directory: {hf_datase
```

```
    shutil.rmtree(hf_dataset_cache_dir)
    print("Hugging Face datasets cache deleted successfully.")
else:
    print(f"\nHugging Face datasets cache directory not found at {hf_dat

# Delete Hub cache
if os.path.exists(hf_hub_cache_dir):
    print(f"\nDeleting Hugging Face Hub cache directory: {hf_hub_cache_d
    shutil.rmtree(hf_hub_cache_dir)
    print("Hugging Face Hub cache deleted successfully.")
else:
    print(f"\nHugging Face Hub cache directory not found at {hf_hub_cach

print("\nDisk usage after deleting all Hugging Face caches:")
total, used, free = shutil.disk_usage('/')
print(f"Total: {total / (1024**3):.2f} GB")
print(f"Used: {used / (1024**3):.2f} GB")
print(f"Free: {free / (1024**3):.2f} GB")
```

```
Disk usage before deleting all Hugging Face caches:
Total: 107.72 GB
Used: 20.89 GB
Free: 86.81 GB

Hugging Face datasets cache directory not found at /root/.cache/huggingfa

Hugging Face Hub cache directory not found at /root/.cache/huggingface/hu

Disk usage after deleting all Hugging Face caches:
Total: 107.72 GB
Used: 20.89 GB
Free: 86.81 GB
```

```
# Unzip data from mounted drive
```

```
pip install ml4floods
```

```
Requirement already satisfied: ml4floods in /usr/local/lib/python3.12/dis
Requirement already satisfied: torch in /usr/local/lib/python3.12/dist-pa
Requirement already satisfied: torchvision in /usr/local/lib/python3.12/d
Requirement already satisfied: pytorch-lightning in /usr/local/lib/python
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-pa
Requirement already satisfied: matplotlib in /usr/local/lib/python3.12/di
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-p
Requirement already satisfied: rasterio in /usr/local/lib/python3.12/dist
Requirement already satisfied: geopandas in /usr/local/lib/python3.12/dis
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-pac
Requirement already satisfied: albumentations in /usr/local/lib/python3.1
Requirement already satisfied: google-cloud-storage in /usr/local/lib/pyt
Requirement already satisfied: fsspec in /usr/local/lib/python3.12/dist-p
Requirement already satisfied: gcsfs in /usr/local/lib/python3.12/dist-pa
Requirement already satisfied: requests-html in /usr/local/lib/python3.12
Requirement already satisfied: earthengine-api in /usr/local/lib/python3.
Requirement already satisfied: mercantile in /usr/local/lib/python3.12/di
Requirement already satisfied: seaborn in /usr/local/lib/python3.12/dist-
Requirement already satisfied: gdown in /usr/local/lib/python3.12/dist-pa
Requirement already satisfied: matplotlib-scalebar in /usr/local/lib/pyth
Requirement already satisfied: georeader-spaceml in /usr/local/lib/python
Requirement already satisfied: huggingface-hub[cli,torch] in /usr/local/l
Requirement already satisfied: scipy>=1.10.0 in /usr/local/lib/python3.12
Requirement already satisfied: PyYAML in /usr/local/lib/python3.12/dist-p
Requirement already satisfied: pydantic>=2.9.2 in /usr/local/lib/python3.
Requirement already satisfied: albucore==0.0.24 in /usr/local/lib/python3
Requirement already satisfied: opencv-python-headless>=4.9.0.80 in /usr/l
Requirement already satisfied: stringzilla>=3.10.4 in /usr/local/lib/pyth
Requirement already satisfied: simsimd>=5.9.2 in /usr/local/lib/python3.1
Requirement already satisfied: google-api-python-client>=1.12.1 in /usr/l
Requirement already satisfied: google-auth>=1.4.1 in /usr/local/lib/pytho
Requirement already satisfied: google-auth-httplib2>=0.0.3 in /usr/local/
Requirement already satisfied: httplib2<1dev,>=0.9.2 in /usr/local/lib/py
Requirement already satisfied: requests in /usr/local/lib/python3.12/dist
Requirement already satisfied: aiohttp!=4.0.0a0,!=4.0.0a1 in /usr/local/l
Requirement already satisfied: decorator>4.1.2 in /usr/local/lib/python3.
Requirement already satisfied: google-auth-oauthlib in /usr/local/lib/pyt
Requirement already satisfied: beautifulsoup4 in /usr/local/lib/python3.1
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist
Requirement already satisfied: pyogrio>=0.7.2 in /usr/local/lib/python3.1
Requirement already satisfied: packaging in /usr/local/lib/python3.12/dis
Requirement already satisfied: pyproj>=3.5.0 in /usr/local/lib/python3.12
Requirement already satisfied: shapely>=2.0.0 in /usr/local/lib/python3.1
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/p
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.1
Requirement already satisfied: click>=3.0 in /usr/local/lib/python3.12/di
Requirement already satisfied: affine in /usr/local/lib/python3.12/dist-p
Requirement already satisfied: attrs in /usr/local/lib/python3.12/dist-pa
Requirement already satisfied: certifi in /usr/local/lib/python3.12/dist-
Requirement already satisfied: cligj>=0.5 in /usr/local/lib/python3.12/di
Requirement already satisfied: pyparsing in /usr/local/lib/python3.12/dis
Requirement already satisfied: google-api-core<3.0.0,>=2.27.0 in /usr/loc
Requirement already satisfied: google-cloud-core<3.0.0,>=2.4.2 in /usr/lo
Requirement already satisfied: google-resumable-media<3.0.0,>=2.7.2 in /u
Requirement already satisfied: google-crc32c<2.0.0,>=1.1.3 in /usr/local/
```

```
WARNING: huggingface-hub 1.11.0 does not provide the extra 'cli'
Requirement already satisfied: hf-xet<2.0.0,>=1.4.3 in /usr/local/lib/pyt
Requirement already satisfied: httpx<1,>=0.23.0 in /usr/local/lib/python3
Requirement already satisfied: typer in /usr/local/lib/python3.12/dist-pa
Requirement already satisfied: typing-extensions>=4.1.0 in /usr/local/lib
Requirement already satisfied: safetensors[torch] in /usr/local/lib/pytho
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.12/dis
Requirement already satisfied: torchmetrics>0.7.0 in /usr/local/lib/pytho
Requirement already satisfied: lightning-utilities>=0.10.0 in /usr/local/
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/di
Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.12
Requirement already satisfied: networkx>=2.5.1 in /usr/local/lib/python3.
Requirement already satisfied: jinja2 in /usr/local/lib/python3.12/dist-p
Requirement already satisfied: pyquery in /usr/local/lib/python3.12/dist-
Requirement already satisfied: fake-useragent in /usr/local/lib/python3.1
Requirement already satisfied: parse in /usr/local/lib/python3.12/dist-pa
Requirement already satisfied: bs4 in /usr/local/lib/python3.12/dist-pack
Requirement already satisfied: w3lib in /usr/local/lib/python3.12/dist-pa
Requirement already satisfied: pyppeteer>=0.0.14 in /usr/local/lib/python
Requirement already satisfied: aiohappyeyeballs>=2.5.0 in /usr/local/lib/
Requirement already satisfied: aiosignal>=1.4.0 in /usr/local/lib/python3
Requirement already satisfied: frozenlist>=1.1.1 in /usr/local/lib/python
Requirement already satisfied: multidict<7.0,>=4.5 in /usr/local/lib/pyth
Requirement already satisfied: propcache>=0.2.0 in /usr/local/lib/python3
Requirement already satisfied: yarl<2.0,>=1.17.0 in /usr/local/lib/python
Requirement already satisfied: googleapis-common-protos<2.0.0,>=1.63.2 in
Requirement already satisfied: protobuf<8.0.0,>=4.25.8 in /usr/local/lib/
Requirement already satisfied: proto-plus<2.0.0,>=1.22.3 in /usr/local/li
Requirement already satisfied: uritemplate<5,>=3.0.1 in /usr/local/lib/py
Requirement already satisfied: pyasn1-modules>=0.2.1 in /usr/local/lib/py
Requirement already satisfied: rsa<5,>=3.1.4 in /usr/local/lib/python3.12
Requirement already satisfied: anyio in /usr/local/lib/python3.12/dist-pa
Requirement already satisfied: httpcore==1.* in /usr/local/lib/python3.12
Requirement already satisfied: idna in /usr/local/lib/python3.12/dist-pac
Requirement already satisfied: h11>=0.16 in /usr/local/lib/python3.12/dis
Requirement already satisfied: annotated-types>=0.6.0 in /usr/local/lib/p
Requirement already satisfied: pydantic-core==2.41.4 in /usr/local/lib/py
Requirement already satisfied: typing-inspection>=0.4.2 in /usr/local/lib
Requirement already satisfied: appdirs<2.0.0,>=1.4.3 in /usr/local/lib/py
Requirement already satisfied: importlib-metadata>=1.4 in /usr/local/lib/
Requirement already satisfied: pyee<12.0.0,>=11.0.0 in /usr/local/lib/pyt
Requirement already satisfied: urllib3<2.0.0,>=1.25.8 in /usr/local/lib/p
Requirement already satisfied: websockets<11.0,>=10.0 in /usr/local/lib/p
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/pytho
Requirement already satisfied: soupsieve>1.2 in /usr/local/lib/python3.12
Requirement already satisfied: requests-oauthlib>=0.7.0 in /usr/local/lib
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.
Requirement already satisfied: lxml>=2.1 in /usr/local/lib/python3.12/dis
Requirement already satisfied: cssselect>=1.2.0 in /usr/local/lib/python3
Requirement already satisfied: PySocks!=1.5.7,>=1.5.6 in /usr/local/lib/p
Requirement already satisfied: shellingham>=1.3.0 in /usr/local/lib/pytho
```

```
Requirement already satisfied: rich>=12.3.0 in /usr/local/lib/python3.12/
Requirement already satisfied: annotated-doc>=0.0.2 in /usr/local/lib/pyt
Requirement already satisfied: zipp>=3.20 in /usr/local/lib/python3.12/di
Requirement already satisfied: p[illegible]/lib/pyt
Requirement already satisfied: oauthlib>=3.0.0 in /usr/local/lib/python3.
Requirement already satisfied: markdown-it-py>=2.0.0 in /usr/local/lib/py
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /usr/local/lib/
Requirement already satisfied: mdurl~=0.1 in /usr/local/lib/python3.12/di
```

Unzipping the WorldFloods Dataset

The following cell will attempt to unzip the `worldfloods_v1_0_sample.zip` file, which is approximately 12.7 GB. This will extract the full dataset to disk. Please ensure you have sufficient disk space, as this operation may consume a significant amount.

```
from ml4floods.models import dataset_setup
import zipfile

# Unzip the data
path_to_dataset_folder = "."
dataset_folder = os.path.join(path_to_dataset_folder, "worldfloods_v1_0_

try:
    dataset_setup.validate_worldfloods_data(dataset_folder)
except FileNotFoundError as e:
    print(e)
    zip_file_name = os.path.join(public_folder, "worldfloods_v1_0_sample

    print("We need to unzip the data")
    # Download the zip file
    if not os.path.exists(zip_file_name):
        print("Download the data from Google Drive")
        import gdown
        # https://drive.google.com/file/d/1106aKZk4R6DERIx32o4mMTJ5dtzRR
        gdown.download(id="1106aKZk4R6DERIx32o4mMTJ5dtzRRKgV", output=zi

    print("Unzipping the data")
    with zipfile.ZipFile(zip_file_name, "r") as zip_ref:
        zip_ref.extractall(path_to_dataset_folder)
        zip_ref.close()
```

```
 Folder not found ./worldfloods_v1_0_sample/train/gt
We need to unzip the data
Unzipping the data
```

```
# View contents of the unzipped folder
!ls -R ./worldfloods_v1_0_sample/
```

```
./worldfloods_v1_0_sample/:
checkpoints  LICENCE-CC-NC.txt   test  train  val

./worldfloods_v1_0_sample/checkpoints:
WF2_unet  WFV1_scnn20  WFV1_unet

./worldfloods_v1_0_sample/checkpoints/WF2_unet:
config.json  model.pt

./worldfloods_v1_0_sample/checkpoints/WFV1_scnn20:
config.json  model.pt

./worldfloods_v1_0_sample/checkpoints/WFV1_unet:
config.json  model.pt

./worldfloods_v1_0_sample/test:
floodmaps  gt  meta  PERMANENTWATERJRC   S2

./worldfloods_v1_0_sample/test/floodmaps:
EMSR286_08ITUANGONORTH_DEL_MONIT02_v1_observed_event_a.cpg
EMSR286_08ITUANGONORTH_DEL_MONIT02_v1_observed_event_a.dbf
EMSR286_08ITUANGONORTH_DEL_MONIT02_v1_observed_event_a.prj
EMSR286_08ITUANGONORTH_DEL_MONIT02_v1_observed_event_a.shp
EMSR286_08ITUANGONORTH_DEL_MONIT02_v1_observed_event_a.shx
EMSR286_09ITUANGOSOUTH_DEL_MONIT02_v1_observed_event_a.cpg
EMSR286_09ITUANGOSOUTH_DEL_MONIT02_v1_observed_event_a.dbf
EMSR286_09ITUANGOSOUTH_DEL_MONIT02_v1_observed_event_a.prj
EMSR286_09ITUANGOSOUTH_DEL_MONIT02_v1_observed_event_a.shp
EMSR286_09ITUANGOSOUTH_DEL_MONIT02_v1_observed_event_a.shx
EMSR333_01RATTALORO_DEL_MONIT01_v1_observed_event_a.cpg
EMSR333_01RATTALORO_DEL_MONIT01_v1_observed_event_a.dbf
EMSR333_01RATTALORO_DEL_MONIT01_v1_observed_event_a.prj
EMSR333_01RATTALORO_DEL_MONIT01_v1_observed_event_a.shp
EMSR333_01RATTALORO_DEL_MONIT01_v1_observed_event_a.shx
EMSR333_02PORTOPALO_DEL_MONIT01_v1_observed_event_a.cpg
EMSR333_02PORTOPALO_DEL_MONIT01_v1_observed_event_a.dbf
EMSR333_02PORTOPALO_DEL_MONIT01_v1_observed_event_a.prj
EMSR333_02PORTOPALO_DEL_MONIT01_v1_observed_event_a.shp
EMSR333_02PORTOPALO_DEL_MONIT01_v1_observed_event_a.shx
EMSR333_13TORRECOLONNASPERONE_DEL_MONIT01_v2_observed_event_a.cpg
EMSR333_13TORRECOLONNASPERONE_DEL_MONIT01_v2_observed_event_a.dbf
EMSR333_13TORRECOLONNASPERONE_DEL_MONIT01_v2_observed_event_a.prj
EMSR333_13TORRECOLONNASPERONE_DEL_MONIT01_v2_observed_event_a.shp
EMSR333_13TORRECOLONNASPERONE_DEL_MONIT01_v2_observed_event_a.shx
EMSR342_06NORTHNORMANTON_DEL_v1_observed_event_a.cpg
EMSR342_06NORTHNORMANTON_DEL_v1_observed_event_a.dbf
EMSR342_06NORTHNORMANTON_DEL_v1_observed_event_a.prj
EMSR342_06NORTHNORMANTON_DEL_v1_observed_event_a.shp
EMSR342_06NORTHNORMANTON_DEL_v1_observed_event_a.shx
EMSR342_07SOUTHNORMANTON_DEL_MONIT03_v2_observed_event_a.cpg
EMSR342_07SOUTHNORMANTON_DEL_MONIT03_v2_observed_event_a.dbf
EMSR342_07SOUTHNORMANTON_DEL_MONIT03_v2_observed_event_a.prj
EMSR342_07SOUTHNORMANTON_DEL_MONIT03_v2_observed_event_a.shp
EMSR342_07SOUTHNORMANTON_DEL_MONIT03_v2_observed_event_a.shx
EMSR347_06MWANZA_DEL_v1_observed_event_a.cpg
EMSR347_06MWANZA_DEL_v1_observed_event_a.dbf
```

```
EMSR347_06MWANZA_DEL_v1_observed_event_a.prj
EMSR347_06MWANZA_DEL_v1_observed_event_a.shp
EMSR347_06MWANZA_DEL_v1_observed_event_a.shx
EMSR347_07ZOMBA_DEL_MONIT01_v1_observed_event_a.cpg
EMSR347_07ZOMBA_DEL_MONIT01_v1_observed_event_a.dbf
EMSR347_07ZOMBA_DEL_MONIT01_v1_observed_event_a.prj
EMSR347_07ZOMBA_DEL_MONIT01_v1_observed_event_a.shp
EMSR347_07ZOMBA_DEL_MONIT01_v1_observed_event_a.shx
EMSR347_07ZOMBA_DEL_v2_observed_event_a.cpg
EMSR347_07ZOMBA_DEL_v2_observed_event_a.dbf
EMSR347_07ZOMBA_DEL_v2_observed_event_a.prj
EMSR347_07ZOMBA_DEL_v2_observed_event_a.shp
EMSR347_07ZOMBA_DEL_v2_observed_event_a.shx
EMSR9284_01YLITORNIONORTHERN_DEL_MONIT01_v1_observed_event_a.cpg
EMSR9284_01YLITORNIONORTHERN_DEL_MONIT01_v1_observed_event_a.dbf
EMSR9284_01YLITORNIONORTHERN_DEL_MONIT01_v1_observed_event_a.prj
EMSR9284_01YLITORNIONORTHERN_DEL_MONIT01_v1_observed_event_a.shp
EMSR9284_01YLITORNIONORTHERN_DEL_MONIT01_v1_observed_event_a.shx

./worldfloods_v1_0_sample/test/gt:
EMSR286_08ITUANGONORTH_DEL_MONIT02_v1_observed_event_a.tif
EMSR286_09ITUANGOSOUTH_DEL_MONIT02_v1_observed_event_a.tif
EMSR333_01RATTALORO_DEL_MONIT01_v1_observed_event_a.tif
EMSR333_02PORTOPALO_DEL_MONIT01_v1_observed_event_a.tif
EMSR333_13TORRECOLONNASPERONE_DEL_MONIT01_v2_observed_event_a.tif
EMSR342_06NORTHNORMANTON_DEL_v1_observed_event_a.tif
EMSR342_07SOUTHNORMANTON_DEL_MONIT03_v2_observed_event_a.tif
EMSR347_06MWANZA_DEL_v1_observed_event_a.tif
EMSR347_07ZOMBA_DEL_MONIT01_v1_observed_event_a.tif
EMSR347_07ZOMBA_DEL_v2_observed_event_a.tif
EMSR9284_01YLITORNIONORTHERN_DEL_MONIT01_v1_observed_event_a.tif

./worldfloods_v1_0_sample/test/meta:
EMSR286_08ITUANGONORTH_DEL_MONIT02_v1_observed_event_a.json
EMSR286_09ITUANGOSOUTH_DEL_MONIT02_v1_observed_event_a.json
EMSR333_01RATTALORO_DEL_MONIT01_v1_observed_event_a.json
EMSR333_02PORTOPALO_DEL_MONIT01_v1_observed_event_a.json
EMSR333_13TORRECOLONNASPERONE_DEL_MONIT01_v2_observed_event_a.json
EMSR342_06NORTHNORMANTON_DEL_v1_observed_event_a.json
EMSR342_07SOUTHNORMANTON_DEL_MONIT03_v2_observed_event_a.json
EMSR347_06MWANZA_DEL_v1_observed_event_a.json
EMSR347_07ZOMBA_DEL_MONIT01_v1_observed_event_a.json
EMSR347_07ZOMBA_DEL_v2_observed_event_a.json
EMSR9284_01YLITORNIONORTHERN_DEL_MONIT01_v1_observed_event_a.json

./worldfloods_v1_0_sample/test/PERMANENTWATERJRC:
EMSR286_08ITUANGONORTH_DEL_MONIT02_v1_observed_event_a.tif
EMSR286_09ITUANGOSOUTH_DEL_MONIT02_v1_observed_event_a.tif
EMSR333_01RATTALORO_DEL_MONIT01_v1_observed_event_a.tif
EMSR333_02PORTOPALO_DEL_MONIT01_v1_observed_event_a.tif
EMSR342_06NORTHNORMANTON_DEL_v1_observed_event_a.tif
EMSR342_07SOUTHNORMANTON_DEL_MONIT03_v2_observed_event_a.tif
EMSR347_06MWANZA_DEL_v1_observed_event_a.tif
EMSR347_07ZOMBA_DEL_MONIT01_v1_observed_event_a.tif
EMSR347_07ZOMBA_DEL_v2_observed_event_a.tif
EMSR9284_01YLITORNIONORTHERN_DEL_MONIT01_v1_observed_event_a.tif
```

```
./worldfloods_v1_0_sample/test/S2:
EMSR286_08ITUANGONORTH_DEL_MONIT02_v1_observed_event_a.tif
EMSR286_09ITUANGOSOUTH_DEL_MONIT02_v1_observed_event_a.tif
EMSR333_01RATTALORO_DEL_MONIT01_v1_observed_event_a.tif
EMSR333_02PORTOPALO_DEL_MONIT01_v1_observed_event_a.tif
EMSR333_13TORRECOLONNASPERONE_DEL_MONIT01_v2_observed_event_a.tif
EMSR342_06NORTHNORMANTON_DEL_v1_observed_event_a.tif
EMSR342_07SOUTHNORMANTON_DEL_MONIT03_v2_observed_event_a.tif
EMSR347_06MWANZA_DEL_v1_observed_event_a.tif
EMSR347_07ZOMBA_DEL_MONIT01_v1_observed_event_a.tif
EMSR347_07ZOMBA_DEL_v2_observed_event_a.tif
EMSR9284_01YLITORNIONORTHERN_DEL_MONIT01_v1_observed_event_a.tif

./worldfloods_v1_0_sample/train:
gt  S2

./worldfloods_v1_0_sample/train/gt:
01042016_Holmes_Creek_at_Vernon_FL.tif
08162016_Mississippi_River_at_Baton_Rouge_LA.tif
EMSR257_01MANDRA_GRA_v2_observed_event_a.tif
EMSR258_03KOLONJE_DEL_v2_observed_event_a.tif
EMSR258_06VLORE_DEL_v2_observed_event_a.tif
EMSR258_07ARMEN_DEL_v2_observed_event_a.tif
EMSR260_03SORBOLO_DEL_MONIT01_v2_observed_event_a.tif
EMSR260_09RUBIERA_DEL_MONIT02_v2_observed_event_a.tif
EMSR261_03WOLFSBURG_DEL_MONIT06_v2_observed_event_a.tif
EMSR280_01MALUNG_DEL_MONIT05_v2_observed_event_a.tif
EMSR283_06GRINDER_DEL_v1_observed_event_a.tif
EMSR287_04PLAUEN_DEL_v1_observed_event_a.tif
EMSR293_01BOTOSANI_DEL_MONIT01_v1_observed_event_a.tif
EMSR293_01BOTOSANI_DEL_v1_observed_event_a.tif
EMSR293_03OZUN_GRA_v1_observed_event_a.tif
EMSR324_04LESPIGNAN_DEL_MONIT01_v1_observed_event_a.tif
EMSR326_02ANTEQUERA_DEL_MONIT01_v1_observed_event_a.tif
EMSR332_01VERONA_DEL_MONIT01_v1_observed_event_a.tif
ST_20170326_WaterExtent_OmusatiAndOshana0000000000-0000012544.tif
ST_20170730_WaterExtent_Champasack_Province0000012544-0000000000.tif

./worldfloods_v1_0_sample/train/S2:
01042016_Holmes_Creek_at_Vernon_FL.tif
08162016_Mississippi_River_at_Baton_Rouge_LA.tif
EMSR257_01MANDRA_GRA_v2_observed_event_a.tif
EMSR258_03KOLONJE_DEL_v2_observed_event_a.tif
EMSR258_06VLORE_DEL_v2_observed_event_a.tif
EMSR258_07ARMEN_DEL_v2_observed_event_a.tif
EMSR260_03SORBOLO_DEL_MONIT01_v2_observed_event_a.tif
EMSR260_09RUBIERA_DEL_MONIT02_v2_observed_event_a.tif
EMSR261_03WOLFSBURG_DEL_MONIT06_v2_observed_event_a.tif
EMSR280_01MALUNG_DEL_MONIT05_v2_observed_event_a.tif
EMSR283_06GRINDER_DEL_v1_observed_event_a.tif
EMSR287_04PLAUEN_DEL_v1_observed_event_a.tif
EMSR293_01BOTOSANI_DEL_MONIT01_v1_observed_event_a.tif
EMSR293_01BOTOSANI_DEL_v1_observed_event_a.tif
EMSR293_03OZUN_GRA_v1_observed_event_a.tif
EMSR324_04LESPIGNAN_DEL_MONIT01_v1_observed_event_a.tif
EMSR326_02ANTEQUERA_DEL_MONIT01_v1_observed_event_a.tif
```

```
EMSR332_01VERONA_DEL_MONIT01_v1_observed_event_a.tif
ST_20170326_WaterExtent_OmusatiAndOshana0000000000-0000012544.tif
ST_20170730_WaterExtent_Champasack_Province0000012544-0000000000.tif

./worldfloods_v1_0_sample/val:
floodmaps  gt  meta  PERMANENTWATERJRC   S2

./worldfloods_v1_0_sample/val/floodmaps:
EMSR271_02FARKADONA_DEL_v1_observed_event_a.cpg
EMSR271_02FARKADONA_DEL_v1_observed_event_a.dbf
EMSR271_02FARKADONA_DEL_v1_observed_event_a.prj
EMSR271_02FARKADONA_DEL_v1_observed_event_a.shp
EMSR271_02FARKADONA_DEL_v1_observed_event_a.shx
EMSR279_05ZARAGOZA_DEL_MONIT02_v1_observed_event_a.cpg
EMSR279_05ZARAGOZA_DEL_MONIT02_v1_observed_event_a.dbf
EMSR279_05ZARAGOZA_DEL_MONIT02_v1_observed_event_a.prj
EMSR279_05ZARAGOZA_DEL_MONIT02_v1_observed_event_a.shp
EMSR279_05ZARAGOZA_DEL_MONIT02_v1_observed_event_a.shx
EMSR280_03FALUN_DEL_MONIT06_v2_observed_event_a.cpg
EMSR280_03FALUN_DEL_MONIT06_v2_observed_event_a.dbf
EMSR280_03FALUN_DEL_MONIT06_v2_observed_event_a.prj
EMSR280_03FALUN_DEL_MONIT06_v2_observed_event_a.shp
EMSR280_03FALUN_DEL_MONIT06_v2_observed_event_a.shx
EMSR287_05MAGWITZ_DEL_v1_observed_event_a.cpg
EMSR287_05MAGWITZ_DEL_v1_observed_event_a.dbf
EMSR287_05MAGWITZ_DEL_v1_observed_event_a.prj
EMSR287_05MAGWITZ_DEL_v1_observed_event_a.shp
EMSR287_05MAGWITZ_DEL_v1_observed_event_a.shx
RS2_20161008_Water_Extent_Corail_Pestel.cpg
RS2_20161008_Water_Extent_Corail_Pestel.dbf
RS2_20161008_Water_Extent_Corail_Pestel.prj
RS2_20161008_Water_Extent_Corail_Pestel.shp
RS2_20161008_Water_Extent_Corail_Pestel.shx
ST1_20161014_WaterExtent_BinhDinh_Lake.cpg
ST1_20161014_WaterExtent_BinhDinh_Lake.dbf
ST1_20161014_WaterExtent_BinhDinh_Lake.prj
ST1_20161014_WaterExtent_BinhDinh_Lake.shp
ST1_20161014_WaterExtent_BinhDinh_Lake.shx

./worldfloods_v1_0_sample/val/gt:
EMSR271_02FARKADONA_DEL_v1_observed_event_a.tif
EMSR279_05ZARAGOZA_DEL_MONIT02_v1_observed_event_a.tif
EMSR280_03FALUN_DEL_MONIT06_v2_observed_event_a.tif
EMSR287_05MAGWITZ_DEL_v1_observed_event_a.tif
RS2_20161008_Water_Extent_Corail_Pestel.tif
ST1_20161014_WaterExtent_BinhDinh_Lake.tif

./worldfloods_v1_0_sample/val/meta:
EMSR271_02FARKADONA_DEL_v1_observed_event_a.json
EMSR279_05ZARAGOZA_DEL_MONIT02_v1_observed_event_a.json
EMSR280_03FALUN_DEL_MONIT06_v2_observed_event_a.json
EMSR287_05MAGWITZ_DEL_v1_observed_event_a.json
RS2_20161008_Water_Extent_Corail_Pestel.json
ST1_20161014_WaterExtent_BinhDinh_Lake.json

./worldfloods_v1_0_sample/val/PERMANENTWATERJRC:
EMSR271_02FARKADONA_DEL_v1_observed_event_a.tif
```

```
# Visualize sample S2 image from train
```

```
S1_20161014_waterExtent_BinhDinh_Lake.tif
```

```
from ml4floods.visualization import plot_utils
import matplotlib.pyplot as plt
from pathlib import Path
import random
import torch
from torch.utils.data import Dataset
import torch.nn.functional as F
import numpy as np
import rasterio
from rasterio.warp import reproject
from rasterio.enums import Resampling

# Assuming BANDS_S2 is defined globally or imported from ml4floods
BANDS_S2 = ['B1','B2','B3','B4','B5','B6','B7','B8','B8A','B9','B10','B1

# Constants for WorldFloods GT structure
LANDCOVER_WATER = 2
QUALITY_CLEAR = 1

class WorldFloodsTilesDataset(Dataset):
    """Reads S2 tiles and aligned GT masks."""

    def __init__(
        self,
        data_root,
        split='train',
        bands=('B4', 'B3', 'B8'),
        max_samples=None,
        transforms=None,
        scale=3500.0,
        target_size=None,
    ):
        self.data_root = Path(data_root)
        self.s2_dir = self.data_root / split / 'S2'
        self.gt_dir = self.data_root / split / 'gt'
        self.transforms = transforms
        self.scale = float(scale)
        self.band_names = list(bands)
        self.band_idxs = [BANDS_S2.index(b) + 1 for b in self.band_names
        self.target_size = tuple(target_size) if target_size is not None

        s2_files = sorted(self.s2_dir.glob('*.tif')) if self.s2_dir.exis
        pairs = []
        for s in s2_files:
            candidate_gt = self.gt_dir / f"{s.stem}.tif"
            if candidate_gt.exists():
```

```
                pairs.append((s, candidate_gt))
            else:
                prefix = s.stem.split('_')[0]
                cand = sorted(self.gt_dir.glob(f"{prefix}*.tif")) if sel
                if cand:
                    pairs.append((s, cand[0]))

        if max_samples is not None:
            pairs = pairs[:max_samples]
        if not pairs:
            raise RuntimeError(f'No S2/GT pairs found in {self.s2_dir} /
        self.pairs = pairs

    def __len__(self):
        return len(self.pairs)

    def _read_gt(self, gt_path):
        with rasterio.open(gt_path) as src_gt:
            meta = {
                'transform': src_gt.transform,
                'crs': src_gt.crs,
                'nodata': src_gt.nodata,
                'shape': (src_gt.height, src_gt.width),
            }
            if src_gt.count == 1:
                landcover = src_gt.read(1).astype('int32')
                quality = np.full(landcover.shape, QUALITY_CLEAR, dtype=
                print(f"Warning: GT file {gt_path} has 1 band. Assuming
            elif src_gt.count >= 2:
                quality = src_gt.read(1).astype('int32')
                landcover = src_gt.read(2).astype('int32')
            else:
                raise RuntimeError(
                    f"Expected at least 1-band GT (landcover) but got {s
                )
        return quality, landcover, meta

    def _align_gt_band(self, band_arr, src_transform, src_crs, dst_shape
        dst = np.full(dst_shape, fill_value, dtype=band_arr.dtype)
        reproject(
            source=band_arr,
            destination=dst,
            src_transform=src_transform,
            src_crs=src_crs,
            dst_transform=dst_transform,
            dst_crs=dst_crs,
            resampling=Resampling.nearest,
        )
        return dst

    def __getitem__(self, idx):
```

```
        s2_path, gt_path = self.pairs[idx]

        with rasterio.open(s2_path) as src_s2:
            bands = [src_s2.read(i).astype('float32') for i in self.band
            s2_crs = src_s2.crs
            s2_transform = src_s2.transform
            height, width = src_s2.height, src_s2.width

        img = np.stack(bands, axis=0) / self.scale

        quality, landcover, gt_meta = self._read_gt(gt_path)

        if gt_meta['shape'] != (height, width) or gt_meta['crs'] != s2_c
            quality = self._align_gt_band(
                quality,
                src_transform=gt_meta['transform'],
                src_crs=gt_meta['crs'],
                dst_shape=(height, width),
                dst_transform=s2_transform,
                dst_crs=s2_crs,
                fill_value=0,
            )
            landcover = self._align_gt_band(
                landcover,
                src_transform=gt_meta['transform'],
                src_crs=gt_meta['crs'],
                dst_shape=(height, width),
                dst_transform=s2_transform,
                dst_crs=s2_crs,
                fill_value=0,
            )

        gt_mask = (landcover == LANDCOVER_WATER).astype('uint8')
        valid = ((quality == QUALITY_CLEAR) & (landcover != 0)).astype('

        if self.transforms is not None:
            try:
                img, gt_mask, valid = self.transforms(img, gt_mask, vali
            except TypeError:
                img, gt_mask = self.transforms(img, gt_mask)

        img_t = torch.from_numpy(img).float()
        mask_t = torch.from_numpy(gt_mask).unsqueeze(0).float()
        valid_t = torch.from_numpy(valid).unsqueeze(0).float()

        if self.target_size is not None:
            H, W = self.target_size
            img_t = F.interpolate(img_t.unsqueeze(0), size=(H, W), mode=
            mask_t = F.interpolate(mask_t.unsqueeze(0), size=(H, W), mod
            valid_t = F.interpolate(valid_t.unsqueeze(0), size=(H, W), m
```

```
        return img_t, mask_t, valid_t

def simple_transforms(img, mask, valid=None, p_flip=0.5):
    if random.random() < p_flip:
        img = img[:, :, ::-1].copy()
        mask = mask[:, ::-1].copy()
        if valid is not None:
            valid = valid[:, ::-1].copy()
    if random.random() < p_flip:
        img = img[:, ::-1, :].copy()
        mask = mask[::-1, :].copy()
        if valid is not None:
            valid = valid[::-1, :].copy()
    if valid is None:
        return img, mask
    return img, mask, valid

# Re-initialize DATA_ROOT and train_ds to ensure it's defined
DATA_ROOT = Path('./worldfloods_v1_0_sample')

# Assuming WorldFloodsTilesDataset and simple_transforms are defined in
train_ds = WorldFloodsTilesDataset(
    data_root=DATA_ROOT,
    split='train',
    bands=('B4', 'B3', 'B8'),
    max_samples=250,
    transforms=simple_transforms,
    target_size=(512, 512),
)

# Get the path to a sample S2 image from the training dataset
sample_s2_path = train_ds.pairs[0][0]

# Manually read the RGB bands from the .tif file
rgb_bands_names = ['B4', 'B3', 'B2'] # Standard RGB for S2
# Using the global BANDS_S2 list from the notebook for band indices
rgb_bands_indices = [BANDS_S2.index(b) + 1 for b in rgb_bands_names] # +

image_data = None
try:
    with rasterio.open(sample_s2_path) as src:
        # Read the selected RGB bands
        # Ensure the image is scaled similarly to how the dataset handle
        image_data = np.stack([src.read(idx) for idx in rgb_bands_indice
except Exception as e:
    print(f"Error reading image with rasterio: {e}")

# Initialize figure
fig, ax = plt.subplots(figsize=(16, 16))

if image_data is not None:
```

```
        # Transpose from (C, H, W) to (H, W, C) for imshow
        img_to_plot = image_data.transpose(1, 2, 0)

        # Apply clipping and normalization as ml4floods plot_rgb_image would
        max_clip_val = 2.5
        min_clip_val = 0.0
        img_to_plot = np.clip(img_to_plot, min_clip_val, max_clip_val)
        img_to_plot = (img_to_plot - min_clip_val) / (max_clip_val - min_cli

        ax.imshow(img_to_plot)
    else:
        ax.text(0.5, 0.5, "Could not load image data.", horizontalalignment=

    ax.set(
        title="Sample Sentinel-2 Image from Training Set"
    )
    ax.axis('off')
    ax.grid(False)
    plt.show()
```

Sample Sentinel-2 Image from Training Set

Standard U-Net

The Architecture: Vanilla U-Net

- Backbone: ResNet-18 or ResNet-34 encoder as it is light to train on my resource constrained computer
- Loss Function: Dice Loss or combination of Binary Cross Entropy + Dice to handle the class imbalance since most of the image is land and a small part is flood

```
# ResNet-backed U-Net (lightweight; supports resnet18 or resnet34 encode
import torch
import torch.nn as nn
import torch.nn.functional as F
import torchvision.models as models


class ConvRelu(nn.Module):
    def __init__(self, in_ch, out_ch, kernel_size=3, padding=1):
        super().__init__()
        self.conv = nn.Conv2d(in_ch, out_ch, kernel_size=kernel_size, pa
        self.bn = nn.BatchNorm2d(out_ch)
        self.relu = nn.ReLU(inplace=True)

    def forward(self, x):
        return self.relu(self.bn(self.conv(x)))


class DecoderBlock(nn.Module):
    def __init__(self, in_ch, skip_ch, out_ch):
        super().__init__()
        # Upsample then convs after concatenation with skip
        self.up = nn.Upsample(scale_factor=2, mode='bilinear', align_cor
        self.conv1 = ConvRelu(in_ch + skip_ch, out_ch)
        self.conv2 = ConvRelu(out_ch, out_ch)

    def forward(self, x, skip):
        x = self.up(x)
        # If necessary, pad to match skip size
        if x.shape[-2:] != skip.shape[-2:]:
            x = F.interpolate(x, size=skip.shape[-2:], mode='bilinear',
        x = torch.cat([x, skip], dim=1)
        x = self.conv1(x)
        x = self.conv2(x)
        return x


class UNetResNet(nn.Module):
    """
```

```
    U-Net with a ResNet encoder (choose depth=18 or 34). Outputs logits
    """
    def __init__(self, encoder='resnet18', pretrained=False, out_channel
        super().__init__()
        assert encoder in ('resnet18', 'resnet34')
        if encoder == 'resnet18':
            resnet = models.resnet18(pretrained=pretrained)
            filters = [64, 64, 128, 256, 512]
        else:
            resnet = models.resnet34(pretrained=pretrained)
            filters = [64, 64, 128, 256, 512]

        # Adapt first conv if input channels != 3
        if n_input_channels != 3:
            old_conv = resnet.conv1
            resnet.conv1 = nn.Conv2d(n_input_channels, old_conv.out_chan
                                     kernel_size=old_conv.kernel_size,
                                     stride=old_conv.stride,
                                     padding=old_conv.padding,
                                     bias=old_conv.bias)

        # Encoder (take sequential layers from ResNet)
        self.initial = nn.Sequential(resnet.conv1, resnet.bn1, resnet.re
        self.pool = resnet.maxpool
        self.encoder1 = resnet.layer1  # 64
        self.encoder2 = resnet.layer2  # 128
        self.encoder3 = resnet.layer3  # 256
        self.encoder4 = resnet.layer4  # 512

        # Center / bridge
        self.center = nn.Sequential(
            ConvRelu(filters[4], filters[4]),
            ConvRelu(filters[4], filters[4])
        )

        # Decoder blocks (match channels to encoder skip connections)
        self.dec4 = DecoderBlock(filters[4], filters[3], filters[3])  #
        self.dec3 = DecoderBlock(filters[3], filters[2], filters[2])  #
        self.dec2 = DecoderBlock(filters[2], filters[1], filters[1])  #
        self.dec1 = DecoderBlock(filters[1], filters[0], filters[0])  #

        # Final conv to output desired channels
        self.final_conv = nn.Sequential(
            ConvRelu(filters[0], filters[0] // 2),
            nn.Conv2d(filters[0] // 2, out_channels, kernel_size=1)
        )

    def forward(self, x):
        # Encoder
        x0 = self.initial(x)      # after conv1: 64 ch, spatial /2
        x1 = self.pool(x0)        # /4
```

```
        e1 = self.encoder1(x1)    # 64
        e2 = self.encoder2(e1)    # 128
        e3 = self.encoder3(e2)    # 256
        e4 = self.encoder4(e3)    # 512

        # Center
        c = self.center(e4)

        # Decoder (upsample + skip connections)
        d4 = self.dec4(c, e3)
        d3 = self.dec3(d4, e2)
        d2 = self.dec2(d3, e1)
        d1 = self.dec1(d2, x0)

        out = self.final_conv(d1)
        # returns logits; apply sigmoid in training/inference as needed
        return out
```

```
# Dataset, DataLoader, training and evaluation for S2 to GT flood masks
import random
from torch.utils.data import Dataset, DataLoader
from pathlib import Path
import torch.nn.functional as F  # needed for interpolation/resizing

# Geospatial reprojection utils
from rasterio.warp import reproject
from rasterio.enums import Resampling

# using local BANDS_S2 defined in the imports cell above instead of impo

# ---- WorldFloods GT structure (as observed in your files) ----
# GT is a 2-band raster:
#   Band 1 (channel 0): Quality / cloud mask
#       0 = no data / invalid
#       1 = clear
#       2 = cloud
#   Band 2 (channel 1): Land cover
#       0 = no data
#       1 = land (dry)
#       2 = water (total water = permanent + flood)
#
# Target definition:
#   - water pixels are landcover == 2
# Validity definition (for loss/metrics masking):
#   - quality == 1 (clear)
#   - and landcover != 0 (not nodata)
LANDCOVER_WATER = 2
QUALITY_CLEAR = 1
```

```
class WorldFloodsTilesDataset(Dataset):
    """Reads S2 tiles and aligned GT masks.

    Returns:
      - img_t: (C,H,W) float32
      - mask_t: (1,H,W) float32 in {0,1} where 1 indicates water (total
      - valid_t: (1,H,W) float32 in {0,1} where 1 indicates pixels to in

    Args:
      - bands: list of band names, e.g. ['B4','B3','B8']
      - max_samples: optional cap for quick tests
      - transforms: callable(img, mask, valid) -> (img, mask, valid) ope
      - target_size: (H, W) tuple to resize all samples to a fixed spati
    """

    def __init__(
        self,
        data_root,
        split='train',
        bands=('B4', 'B3', 'B8'),
        max_samples=None,
        transforms=None,
        scale=3500.0,
        target_size=None,
    ):
        self.data_root = Path(data_root)
        self.s2_dir = self.data_root / split / 'S2'
        self.gt_dir = self.data_root / split / 'gt'
        self.transforms = transforms
        self.scale = float(scale)
        self.band_names = list(bands)
        # map band names to 1-based rasterio indexes using local BANDS_S
        self.band_idxs = [BANDS_S2.index(b) + 1 for b in self.band_names
        self.target_size = tuple(target_size) if target_size is not None

        s2_files = sorted(self.s2_dir.glob('*.tif')) if self.s2_dir.exis
        pairs = []
        for s in s2_files:
            candidate_gt = self.gt_dir / f"{s.stem}.tif"
            if candidate_gt.exists():
                pairs.append((s, candidate_gt))
            else:
                prefix = s.stem.split('_')[0]
                cand = sorted(self.gt_dir.glob(f"{prefix}*.tif")) if sel
                if cand:
                    pairs.append((s, cand[0]))

        if max_samples is not None:
            pairs = pairs[:max_samples]
        if not pairs:
```

```
            raise RuntimeError(f'No S2/GT pairs found in {self.s2_dir} /
        self.pairs = pairs

    def __len__(self):
        return len(self.pairs)

    def _read_gt(self, gt_path):
        """Read GT quality + landcover bands.

        Returns:
          quality: (H,W) int32
          landcover: (H,W) int32
          meta: dict with transform/crs/nodata/shape
        """
        with rasterio.open(gt_path) as src_gt:
            meta = {
                'transform': src_gt.transform,
                'crs': src_gt.crs,
                'nodata': src_gt.nodata,
                'shape': (src_gt.height, src_gt.width),
            }
            if src_gt.count == 1:
                # Assume single band is landcover, create a 'clear' qual
                landcover = src_gt.read(1).astype('int32')
                quality = np.full(landcover.shape, QUALITY_CLEAR, dtype=
                print(f"Warning: GT file {gt_path} has 1 band. Assuming
            elif src_gt.count >= 2:
                # Normal 2-band case
                quality = src_gt.read(1).astype('int32')
                landcover = src_gt.read(2).astype('int32')
            else:
                raise RuntimeError(
                    f"Expected at least 1-band GT (landcover) but got {s
                )
        return quality, landcover, meta

    def _align_gt_band(self, band_arr, src_transform, src_crs, dst_shape
        """Reproject a single-band integer raster onto the S2 grid."""
        dst = np.full(dst_shape, fill_value, dtype=band_arr.dtype)
        reproject(
            source=band_arr,
            destination=dst,
            src_transform=src_transform,
            src_crs=src_crs,
            dst_transform=dst_transform,
            dst_crs=dst_crs,
            resampling=Resampling.nearest,
        )
        return dst

    def __getitem__(self, idx):
```

```
        s2_path, gt_path = self.pairs[idx]

        with rasterio.open(s2_path) as src_s2:
            # Read selected bands
            bands = [src_s2.read(i).astype('float32') for i in self.band
            s2_crs = src_s2.crs
            s2_transform = src_s2.transform
            height, width = src_s2.height, src_s2.width

        # Stack and scale to [0,1] like reflectance
        img = np.stack(bands, axis=0) / self.scale  # shape (C,H,W)

        # Read GT two-band structure
        quality, landcover, gt_meta = self._read_gt(gt_path)

        # Align BOTH quality and landcover to S2 grid if needed
        if gt_meta['shape'] != (height, width) or gt_meta['crs'] != s2_c
            quality = self._align_gt_band(
                quality,
                src_transform=gt_meta['transform'],
                src_crs=gt_meta['crs'],
                dst_shape=(height, width),
                dst_transform=s2_transform,
                dst_crs=s2_crs,
                fill_value=0,
            )
            landcover = self._align_gt_band(
                landcover,
                src_transform=gt_meta['transform'],
                src_crs=gt_meta['crs'],
                dst_shape=(height, width),
                dst_transform=s2_transform,
                dst_crs=s2_crs,
                fill_value=0,
            )

        # Binary water mask from landcover
        gt_mask = (landcover == LANDCOVER_WATER).astype('uint8')  # 1=wa

        # Valid pixels for loss/metrics: clear + landcover not nodata
        valid = ((quality == QUALITY_CLEAR) & (landcover != 0)).astype('

        # Optional simple augmentations (operate on numpy arrays)
        if self.transforms is not None:
            # Support both legacy (img, mask) and (img, mask, valid)
            try:
                img, gt_mask, valid = self.transforms(img, gt_mask, vali
            except TypeError:
                img, gt_mask = self.transforms(img, gt_mask)

        # Convert to torch tensors
```

```
        img_t = torch.from_numpy(img).float()
        mask_t = torch.from_numpy(gt_mask).unsqueeze(0).float()   # (1,H
        valid_t = torch.from_numpy(valid).unsqueeze(0).float()    # (1,H

        # If requested, resize to target_size using torch's interpolate
        if self.target_size is not None:
            H, W = self.target_size
            img_t = F.interpolate(img_t.unsqueeze(0), size=(H, W), mode=
            # mask/valid use nearest interpolation
            mask_t = F.interpolate(mask_t.unsqueeze(0), size=(H, W), mod
            valid_t = F.interpolate(valid_t.unsqueeze(0), size=(H, W), m

        return img_t, mask_t, valid_t


# Simple deterministic transforms (random flips)
def simple_transforms(img, mask, valid=None, p_flip=0.5):
    # img: numpy (C,H,W), mask: (H,W), valid: (H,W) or None
    if random.random() < p_flip:
        # horizontal
        img = img[:, :, ::-1].copy()
        mask = mask[:, ::-1].copy()
        if valid is not None:
            valid = valid[:, ::-1].copy()
    if random.random() < p_flip:
        # vertical
        img = img[:, ::-1, :].copy()
        mask = mask[::-1, :].copy()
        if valid is not None:
            valid = valid[::-1, :].copy()

    if valid is None:
        return img, mask
    return img, mask, valid
```

```
class HuggingFaceWorldFloodsDataset(Dataset):
    """Reads S2 tiles and aligned GT masks from a Hugging Face dataset.

    Returns:
      - img_t: (C,H,W) float32
      - mask_t: (1,H,W) float32 in {0,1} where 1 indicates water (total
      - valid_t: (1,H,W) float32 in {0,1} where 1 indicates pixels to in

    Args:
      - hf_dataset: A Hugging Face dataset object (e.g., hf_dataset_trai
      - bands: list of band names, e.g. ['B4','B3','B8']
      - max_samples: optional cap for quick tests
      - transforms: callable(img, mask, valid) -> (img, mask, valid) ope
      - target_size: (H, W) tuple to resize all samples to a fixed spati
```

```
    """

    def __init__(
        self,
        hf_dataset,
        bands=('B4', 'B3', 'B8'),
        max_samples=None,
        transforms=None,
        scale=3500.0,
        target_size=None,
    ):
        self.hf_dataset = hf_dataset
        self.transforms = transforms
        self.scale = float(scale)
        self.band_names = list(bands)
        # map band names to 0-based index for reading from HF 'image' li
        self.band_idxs_hf = [BANDS_S2.index(b) for b in self.band_names]
        self.target_size = tuple(target_size) if target_size is not None

        if max_samples is not None:
            self.hf_dataset = self.hf_dataset.select(range(max_samples))

    def __len__(self):
        return len(self.hf_dataset)

    def __getitem__(self, idx):
        item = self.hf_dataset[idx]

        # Extract S2 image data. HF dataset stores bands as a list of ar
        # Each array is (H,W). We need to stack them to (C,H,W)
        bands_data = [item['image'][i] for i in self.band_idxs_hf]
        img = np.stack(bands_data, axis=0).astype('float32') / self.scal

        # Extract GT quality and landcover
        # HF dataset 'gt' field is a list of two arrays: [quality_band,
        quality = np.array(item['gt'][0]).astype('int32') # (H,W)
        landcover = np.array(item['gt'][1]).astype('int32') # (H,W)

        # Binary water mask from landcover
        gt_mask = (landcover == LANDCOVER_WATER).astype('uint8')  # 1=wa

        # Valid pixels for loss/metrics: clear + landcover not nodata
        valid = ((quality == QUALITY_CLEAR) & (landcover != 0)).astype('

        # Optional simple augmentations (operate on numpy arrays)
        if self.transforms is not None:
            try:
                img, gt_mask, valid = self.transforms(img, gt_mask, vali
            except TypeError:
                img, gt_mask = self.transforms(img, gt_mask)
```

```
        # Convert to torch tensors
        img_t = torch.from_numpy(img).float()
        mask_t = torch.from_numpy(gt_mask).unsqueeze(0).float()   # (1,H
        valid_t = torch.from_numpy(valid).unsqueeze(0).float()    # (1,H

        # If requested, resize to target_size using torch's interpolate
        if self.target_size is not None:
            H, W = self.target_size
            img_t = F.interpolate(img_t.unsqueeze(0), size=(H, W), mode=
            # mask/valid use nearest interpolation
            mask_t = F.interpolate(mask_t.unsqueeze(0), size=(H, W), mod
            valid_t = F.interpolate(valid_t.unsqueeze(0), size=(H, W), m

        return img_t, mask_t, valid_t
```

```
# Build dataset from local unzipped files
# Use a fixed target_size so all samples have the same spatial dimension
DATA_ROOT = Path('./worldfloods_v1_0_sample')

train_full_ds = WorldFloodsTilesDataset(
    data_root=DATA_ROOT,
    split='train',
    bands=('B4', 'B3', 'B8'),
    max_samples=250, # Using a subset for faster training
    transforms=simple_transforms,
    target_size=(512, 512),
)

val_full_ds = WorldFloodsTilesDataset(
    data_root=DATA_ROOT,
    split='val',
    bands=('B4', 'B3', 'B8'),
    # max_samples=300, # Can set a limit for validation as well if neede
    transforms=simple_transforms,
    target_size=(512, 512),
)

# We now have separate local datasets
train_ds = train_full_ds
val_ds = val_full_ds

# DataLoaders (small batch size for laptop)
batch_size = 2  # reduced to lower memory usage
train_loader = DataLoader(train_ds, batch_size=batch_size, shuffle=True,
val_loader = DataLoader(val_ds, batch_size=batch_size, shuffle=False, nu

print('Total train samples:', len(train_ds), 'Total val samples:', len(v
```

```
# Training utilities: masked BCE + masked dice, masked metrics

def masked_bce_with_logits(logits, targets, valid, eps=1e-6):
    """BCE over valid pixels only.

    logits/targets/valid: (B,1,H,W)
    valid is expected to be 0/1.
    """
    loss_map = F.binary_cross_entropy_with_logits(logits, targets, reduc
    valid = (valid > 0).float()
    denom = valid.sum().clamp_min(eps)
    return (loss_map * valid).sum() / denom


def masked_dice_loss_logits(logits, targets, valid, eps=1e-6):
    """Soft dice over valid pixels only."""
    probs = torch.sigmoid(logits)
    valid = (valid > 0).float()

    probs = probs.view(probs.size(0), -1)
    targets = targets.view(targets.size(0), -1)

    intersection = (probs * targets).sum(dim=1)
    union = probs.sum(dim=1) + targets.sum(dim=1)
    dice = (2 * intersection + eps) / (union + eps)
    return 1.0 - dice.mean()


def threshold_predictions(logits, thresh=0.5):
    return (torch.sigmoid(logits) >= thresh).to(torch.uint8)


def masked_binary_metrics(preds, targets, valid, eps=1e-6):
    """IoU/precision/recall computed over valid pixels only.

    preds/targets/valid: numpy arrays in shape (H,W) with {0,1}
    """
    v = valid.astype(bool).ravel()
    if v.sum() == 0:
        return np.nan, np.nan, np.nan

    y_pred = preds.ravel()[v]
    y_true = targets.ravel()[v]

    tp = int(((y_true == 1) & (y_pred == 1)).sum())
    fp = int(((y_true == 0) & (y_pred == 1)).sum())
    fn = int(((y_true == 1) & (y_pred == 0)).sum())

    prec = tp / (tp + fp + eps)
    rec = tp / (tp + fn + eps)
```

```
    inter = tp
    union = int(((y_true == 1) | (y_pred == 1)).sum())
    iou = inter / (union + eps)
    return iou, prec, rec
```

```
Total train samples: 20 Total val samples: 6
```

```
# Device
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print('Using device:', device)

# Build model
model = UNetResNet(encoder='resnet18', pretrained=False, out_channels=1,
model = model.to(device)
optimizer = torch.optim.Adam(model.parameters(), lr=1e-4)


def _unpack_batch(batch):
    """Support (imgs, masks) and (imgs, masks, valid) batches."""
    if isinstance(batch, (list, tuple)) and len(batch) == 2:
        imgs, masks = batch
        valid = None
        return imgs, masks, valid
    if isinstance(batch, (list, tuple)) and len(batch) == 3:
        imgs, masks, valid = batch
        return imgs, masks, valid
    raise ValueError(
        f"Unexpected batch structure: type={type(batch)} len={len(batch)
    )


def _ensure_valid(valid, like_tensor):
    """If a dataset doesn't provide valid, treat all pixels as valid."""
    if valid is None:
        return torch.ones_like(like_tensor)
    return valid


# Small training loop
num_epochs = 3
for epoch in range(1, num_epochs + 1):
    model.train()
    running_loss = 0.0

    for batch in train_loader:
        imgs, masks, valid = _unpack_batch(batch)
        imgs = imgs.to(device)
        masks = masks.to(device)
        valid = _ensure_valid(valid, masks).to(device)

        optimizer.zero_grad()
```

```
        logits = model(imgs)

        # Resize logits to match GT spatial size before computing losses
        if logits.shape[-2:] != masks.shape[-2:]:
            logits = F.interpolate(logits, size=masks.shape[-2:], mode='

        loss_bce = masked_bce_with_logits(logits, masks, valid)
        loss_dice = masked_dice_loss_logits(logits, masks, valid)
        loss = loss_bce + loss_dice

        loss.backward()
        optimizer.step()
        running_loss += loss.item() * imgs.size(0)

    epoch_loss = running_loss / len(train_ds)

    # Validation (masked metrics)
    model.eval()
    iou_list = []
    prec_list = []
    rec_list = []

    with torch.no_grad():
        for batch in val_loader:
            imgs, masks, valid = _unpack_batch(batch)
            imgs = imgs.to(device)
            masks = masks.to(device)
            valid = _ensure_valid(valid, masks).to(device)

            logits = model(imgs)
            if logits.shape[-2:] != masks.shape[-2:]:
                logits = F.interpolate(logits, size=masks.shape[-2:], mo

            preds = threshold_predictions(logits, thresh=0.5).float()

            preds_np = preds.cpu().numpy().astype(np.uint8)
            masks_np = masks.cpu().numpy().astype(np.uint8)
            valid_np = valid.cpu().numpy().astype(np.uint8)

            for i in range(preds_np.shape[0]):
                iou, prec, rec = masked_binary_metrics(
                    preds_np[i, 0],
                    masks_np[i, 0],
                    valid_np[i, 0],
                )
                if not np.isnan(iou):
                    iou_list.append(iou)
                if not np.isnan(prec):
                    prec_list.append(prec)
                if not np.isnan(rec):
                    rec_list.append(rec)
```

```
        mean_iou = float(np.mean(iou_list)) if iou_list else float('nan')
        mean_prec = float(np.mean(prec_list)) if prec_list else float('nan')
        mean_rec = float(np.mean(rec_list)) if rec_list else float('nan')

        print(
            f'Epoch {epoch}/{num_epochs} | train_loss={epoch_loss:.4f} | '
            f'val IoU={mean_iou:.4f} | prec={mean_prec:.4f} | rec={mean_rec:
        )

    # Save a checkpoint
    ckpt_path = Path('unet_resnet18_checkpoint.pth')
    torch.save({'model_state_dict': model.state_dict(), 'optimizer_state_dic
    print('Saved checkpoint to', ckpt_path)
```